

Natural Language Processing

Course Assignment

From Text Statistics to Transformers: A Full-Spectrum NLP Assignment

Dataset	NLP Learning Dataset (Kaggle)
Dataset URL	https://www.kaggle.com/datasets/engraamirjamil/nlp-learning-dataset
Files Used	All 15 CSV files in the dataset
Level	Intermediate (assumes working Python & basic ML/scikit-learn knowledge)
Format	Jupyter Notebook (.ipynb) with code + written answers in Markdown cells
Total Parts	15 topic parts + 1 capstone integration task

Instructions & Setup

Download the dataset from Kaggle: <https://www.kaggle.com/datasets/engraamirjamil/nlp-learning-dataset>. It contains 15 CSV files, each focused on a different NLP concept, plus a `README.md` describing every column and a `dataset_summary.csv` overview file. Read the README before you begin — it explains exactly what each column means.

General Requirements

- Work in a single Jupyter Notebook (or one notebook per part, your instructor's choice) using Python 3.
- Every coding question must include working, runnable code **and** a short written interpretation of the result (1-3 sentences) in a Markdown cell directly below the code — code with no explanation will not receive full credit.
- All plots must have a title, axis labels, and (where relevant) a legend.
- Set a random seed (e.g. `random_state=42`) wherever randomness is involved, so your results are reproducible.
- You may use pandas, NumPy, scikit-learn, NLTK, spaCy, Gensim, Matplotlib/Seaborn, and Hugging Face Transformers. Cite any other library you use.
- Do not hand-label or manually override any output to make a metric look better than it is. Report your real results, even if a model performs poorly — then explain *why* you think that happened.

Grading Overview

Component	Weight
Parts 1-7 (Foundations: text stats through NER)	25%
Parts 8-11 (Classification, similarity, embeddings, transformers)	25%
Parts 12-15 (QA, summarization, spam detection, topic modeling, language ID)	30%
Capstone Integration Task	15%
Code quality, reproducibility, and written interpretation	5%

Note on data: every file in this dataset is **synthetically generated** for educational purposes. This is a great opportunity to observe how models behave on clean, templated text — pay attention to questions that ask you to compare this behavior with what you'd expect on messy, real-world text.

Assignment Parts

Part 1: Text Statistics & Basic NLP Properties

Dataset file(s): `nlp_basics.csv` | Points: 15

This file contains 2,000 sentences with pre-computed statistics (word count, character count, uppercase count, punctuation count, and a sentence-length category). You will verify these statistics yourself and explore what they reveal about writing style.

- Q1.1.** Load the dataset and, for five randomly sampled rows, recompute `word_count`, `character_count`, `uppercase_count`, and `punctuation_count` from the raw `text` column using your own code (do not just read the existing columns). Confirm your values match the dataset.
- Q1.2.** Plot a histogram of `word_count` across the full dataset. Overlay the mean and median as vertical lines. Is the distribution symmetric or skewed? Explain why.
- Q1.3.** Group the data by `sentence_length` (Short/Medium/Long) and compute the average `punctuation_count` and `uppercase_count` per group. Is there a relationship between sentence length and punctuation density? Support your answer with a bar chart.
- Q1.4.** Write a function `text_stats(text: str) -> dict` that computes all four statistics for an arbitrary input string (not from the dataset). Test it on at least 3 sentences you write yourself, including one written in ALL CAPS and one with heavy punctuation (e.g. "Wait... really?! That's amazing!!!!").
- Q1.5.** Critical thinking: these four statistics are simple, but they are still used as features in real-world spam and abuse-detection systems. Propose one additional statistic (not in the dataset) that you think would help distinguish normal writing from spam or abusive text, and explain your reasoning in 3-5 sentences.

Part 2: Text Preprocessing Pipeline

Dataset file(s): `text_preprocessing.csv` | Points: 20

This file pairs 2,000 raw, messy sentences with each stage of a standard cleaning pipeline: lowercasing, punctuation/number removal, tokenization, stopword removal, stemming, and lemmatization.

- Q2.1.** For 5 sample rows, print the sentence at every pipeline stage (`original_text` → `lowercase_text` → `clean_text` → `tokenized_text` → `stopwords_removed` → `stemmed_text` → `lemmatized_text`) so the transformation is clearly visible step by step.
- Q2.2.** Using NLTK (or spaCy), build your **own** preprocessing pipeline from scratch (`lowercase` → `remove punctuation/numbers` → `tokenize` → `remove stopwords` → `lemmatize`) and apply it to the `original_text` column. Compare your lemmatized output to the dataset's `lemmatized_text` column for 10 rows. Do they match exactly? Explain any differences you find.
- Q2.3.** Explain, in your own words, the difference between stemming and lemmatization. Then find (by comparing columns) at least 2 examples in the dataset where `stemmed_text` and `lemmatized_text` differ for the same word, and explain why.
- Q2.4.** Compute the average number of tokens removed per sentence by stopword removal (i.e. token count before minus after). Plot the distribution. What percentage of the average sentence, by token

count, is typically made up of stopwords in this dataset?

Q2.5. Design an experiment: train two identical text classifiers (see Part 7) — one using raw `original_text` and one using fully preprocessed text — on any labeled dataset of your choice from this collection. Compare accuracy. Does preprocessing help, hurt, or make no difference? Justify your interpretation.

Part 3: Bag of Words & TF-IDF

Dataset file(s): `bow_tfidf.csv` | Points: 15

2,000 sentences across 8 categories, each labeled with the terms a Bag-of-Words model would flag versus the more distinctive terms TF-IDF would highlight.

Q3.1. Using scikit-learn's `CountVectorizer`, build a Bag-of-Words matrix from the `text` column. Report the vocabulary size and the 15 most frequent terms overall.

Q3.2. Using `TfidfVectorizer`, build a TF-IDF matrix from the same text. For 3 sample documents, print the top 5 highest-weighted TF-IDF terms and compare them to the dataset's `tfidf_relevant_terms` column for those rows. Do they align conceptually?

Q3.3. Pick any two categories (e.g. Technology and Sports). For each, compute the average TF-IDF weight of the top 10 terms unique to that category. Explain, in 2-3 sentences, why TF-IDF gives these terms high weight while giving common words like "the" or "company" low weight.

Q3.4. In your own words (no copy-pasted definitions), explain why Bag-of-Words treats the sentences "the dog bit the man" and "the man bit the dog" identically, and why this can be a problem. Name one NLP technique (from any later part of this assignment) that addresses this limitation.

Part 4: N-Grams (Unigrams, Bigrams, Trigrams)

Dataset file(s): `ngrams.csv` | Points: 12

2,000 sentences with pre-computed unigram, bigram, and trigram lists across 8 categories.

Q4.1. Write your own function to generate bigrams and trigrams from raw text (do not use the pre-computed columns), and verify your output matches the dataset's `bigrams` and `trigrams` columns for 5 sample rows.

Q4.2. Across the whole dataset, find the 10 most frequent bigrams and the 10 most frequent trigrams. Display them as a horizontal bar chart, sorted by frequency.

Q4.3. Compare unigram, bigram, and trigram vocabulary sizes (i.e. the number of *distinct* n-grams of each type) across the full corpus. Explain, using your results, why n-gram vocabulary tends to grow rapidly as n increases (this is related to a concept called the "curse of dimensionality").

Q4.4. Pick one category from the dataset. Identify 3 bigrams or trigrams that are highly characteristic of that category (i.e. rarely appear in other categories) and explain how this kind of analysis could be used as a simple category-classification feature.

Part 5: Sentiment Analysis & Emotion Detection

Dataset file(s): `sentiment_analysis.csv` | Points: 20

3,000 review-style texts labeled with sentiment (Positive/Negative/Neutral), a numeric sentiment score, an emotion label, and a category.

Q5.1. Plot the distribution of the `sentiment` column and the `emotion` column. Is the dataset balanced across sentiment classes? Is every emotion equally likely within each sentiment class? Support your answer with a cross-tabulation (`pd.crosstab`) or a stacked bar chart of sentiment vs. emotion.

Q5.2. Split the data into train/test sets (80/20, stratified by sentiment). Train a TF-IDF + Logistic Regression classifier to predict sentiment from text. Report accuracy, precision, recall, and F1-score per class, and show a confusion matrix.

Q5.3. Repeat the classification task from Q5.2 using a different algorithm (e.g. Multinomial Naive Bayes or a linear SVM). Compare performance to Logistic Regression in a small results table and explain which model performed better and why you think that is.

Q5.4. Find and print 5 misclassified examples from your best model. For each, give a plausible explanation for why the model got it wrong (e.g. ambiguous wording, sarcasm, mixed sentiment).

Q5.5. *Optional (bonus, +3%):* Using Hugging Face's transformers pipeline("sentiment-analysis"), run inference on the same test set and compare its accuracy against your TF-IDF model. Discuss the trade-offs (speed, accuracy, resource requirements) between the two approaches.

Part 6: Named Entity Recognition (NER)

Dataset file(s): `named_entity_recognition.csv` | Points: 18

2,000 sentences containing fictional entities (people, organizations, locations, dates, money, products), with entities and types extracted into structured columns. Note: not every sentence contains every entity type — a blank column means that entity type is absent.

Q6.1. For 5 sample rows, print the sentence alongside its `entities` and `entity_types` columns in an aligned, readable format (e.g. PERSON: Maria Chen).

Q6.2. Load spaCy's small English model and run it on 20 sample sentences from the dataset. Compare spaCy's predicted entities and types against the dataset's ground-truth columns for those 20 rows. Report how many entities spaCy correctly identified, missed, or mislabeled (note that spaCy's label names, e.g. GPE, ORG, may differ slightly from this dataset's — map them appropriately before comparing).

Q6.3. Compute how often each entity type (PERSON, ORG, GPE, DATE, MONEY, PRODUCT) appears across the whole dataset. Plot the result as a bar chart. Which entity type is most common, and why do you think that is given how the sentences were generated?

Q6.4. Write a simple rule-based entity extractor using regular expressions that can detect at least the MONEY entity type (e.g. patterns like "\$199.99"). Test it against the dataset's money column and report your precision and recall.

Q6.5. Discuss (5-6 sentences): what are the main limitations of a rule-based or purely statistical NER system compared to a neural/transformer-based NER system, especially when applied to real-world text (e.g. social media posts) rather than the clean, templated sentences in this dataset?

Part 7: Multi-Class Text Classification

Dataset file(s): `text_classification.csv` | Points: 20

3,000 news-style sentences spread across 10 categories and subcategories, with a difficulty rating (Beginner/Intermediate/Advanced) for each row.

Q7.1. Plot the distribution of category and confirm the classes are balanced. Then plot the distribution of difficulty within each category.

Q7.2. Build a full scikit-learn Pipeline (TF-IDF vectorizer + classifier of your choice) to predict category from text. Use 5-fold cross-validation and report the mean and standard deviation of accuracy across folds.

Q7.3. Using your trained model from Q7.2, generate and interpret a full classification report (precision/recall/F1 per class) on a held-out test set. Which category is hardest for the model to predict correctly, and why might that be, based on the topic overlap between categories?

Q7.4. Filter the test set to only `difficulty == "Advanced"` rows. Recompute accuracy on this subset only, and compare it to accuracy on "Beginner" rows. Does model performance vary meaningfully by difficulty level? Explain your result.

Q7.5. Extract and display the top 10 most important TF-IDF features (by model coefficient or feature importance) for any 2 categories of your choice. Do these features make intuitive sense given the category?

Part 8: Sentence Similarity & Semantic Relationships

Dataset file(s): `text_similarity.csv` | Points: 18

2,000 sentence pairs labeled with a similarity score (0-1), a similarity label (Very Low to Very High), and a relationship type (Same Meaning, Paraphrase, Related, Contradiction, Unrelated).

Q8.1. Using TF-IDF vectors and cosine similarity, compute your own similarity score for every sentence pair in the dataset. Compare your computed scores against the dataset's `similarity_score` column using a scatter plot (your score on the x-axis, dataset score on the y-axis) and report the Pearson correlation coefficient.

Q8.2. Group your TF-IDF-based similarity scores by the dataset's `relationship` column (Same Meaning, Paraphrase, Related, Contradiction, Unrelated) using a boxplot. Does TF-IDF cosine similarity correctly rank these relationship types from most to least similar? Where does it struggle, and why?

Q8.3. Contradiction pairs (e.g. "The team won the championship" vs. "The team lost the championship") often confuse simple vector-space models. Find 3 contradiction pairs from the dataset, compute their TF-IDF cosine similarity, and explain in your own words why a purely word-overlap-based method cannot reliably distinguish a contradiction from a paraphrase.

Q8.4. Optional (bonus, +3%): If you have access to a pretrained sentence-embedding model (e.g. via sentence-transformers or a Hugging Face embedding model), repeat Q8.1 using embedding cosine similarity instead of TF-IDF, and compare the correlation with the dataset's ground truth. Which approach correlates more strongly, and why would you expect that?

Part 9: Word Embeddings & Semantic Relationships

Dataset file(s): `word_embeddings.csv` | Points: 15

2,000 word pairs spanning Synonym, Antonym, Related Concept, Category Member, Profession, Location, Object, and Action relationships across 7 vocabulary domains, with an approximate similarity score.

Q9.1. Plot the average `similarity_score` grouped by `relationship` type as a bar chart. Does the ordering match your intuition (e.g. should Synonym pairs have a higher average score than Antonym pairs)? Explain any surprises.

Q9.2. Using Gensim, train a Word2Vec (or FastText) model on a text corpus of your choice from this dataset collection (e.g. concatenate the `text` columns from several files). For 10 word pairs from `word_embeddings.csv`, look up your trained model's cosine similarity and compare it to the dataset's `similarity_score`.

Q9.3. Using your trained embeddings from Q9.2, pick 3 words from the dataset and find each word's 5 nearest neighbors in your embedding space (`model.wv.most_similar()`). Are the neighbors semantically sensible? What limitations do you notice, and how do they relate to the size of your training corpus?

Q9.4. Explain, in your own words, why word embeddings capture analogies such as "king - man + woman $\approx$ queen", and why this property does not emerge from Bag-of-Words or TF-IDF representations.

Part 10: Transformer-Era NLP Tasks

Dataset file(s): `transformer_examples.csv` | Points: 15

2,000 examples evenly covering 8 modern NLP tasks (Sentiment Analysis, Text Classification, NER, Question Answering, Text Summarization, Text Similarity, Text Generation, Masked Language Modeling), each with an expected output and difficulty rating.

Q10.1. Group the dataset by task and, for each task, print 2 example rows showing `text`, `input_type`, and `expected_output`. Briefly describe, in your own words, what makes the input format for each task different (e.g. why does Question Answering need two pieces of text while Sentiment Analysis needs only one).

Q10.2. Filter to rows where `task == "Masked Language Modeling"`. Using Hugging Face's `pipeline("fill-mask")` with a pretrained model (e.g. `bert-base-uncased`), run inference on 10 of these examples and compare the model's top prediction to the dataset's `expected_output`. Report your accuracy.

Q10.3. Choose any 2 other tasks present in this file and, using an appropriate Hugging Face pipeline for each, run inference on 10 examples per task. Report qualitative results (does the output look reasonable?) since exact-match accuracy may not always apply.

Q10.4. Write a short reflection (6-8 sentences) comparing the classical approaches you used earlier in this assignment (TF-IDF, Bag-of-Words, Word2Vec) to the transformer-based approaches used in this part. What capabilities do transformers add? What costs (compute, data, latency) come with those capabilities?

Part 11: Extractive Question Answering

Dataset file(s): [question_answering.csv](#) | Points: 18

2,000 context/question/answer triples where every answer is a verified exact substring of its context, with a character offset (`answer_start`).

Q11.1. For 5 sample rows, verify programmatically that `context[answer_start : answer_start + len(answer)] == answer` holds true. Then intentionally construct one example where this would fail, and explain what type of data error would cause such a mismatch in a real-world QA dataset.

Q11.2. Using a Hugging Face pipeline("question-answering") with a pretrained model, run inference on 20 sample (context, question) pairs. Compare the model's predicted answer to the ground-truth answer column using exact match and partial (substring) match. Report both metrics.

Q11.3. Find 3 examples where the model's answer does not exactly match the ground truth. For each, assess whether the model's answer is still *reasonable* (e.g. off by a word or two, or a synonym) versus genuinely incorrect, and explain your reasoning.

Q11.4. Group the dataset by `difficulty` and compute your model's exact-match accuracy separately for Beginner, Intermediate, and Advanced questions. Does accuracy decrease as difficulty increases? Discuss what makes a question "harder" from a model's perspective.

Part 12: Text Summarization

Dataset file(s): [text_summarization.csv](#) | Points: 18

2,000 short articles paired with concise one-sentence summaries across 8 topic categories.

Q12.1. Implement a simple **extractive** baseline summarizer: for each article, select the single sentence with the highest average TF-IDF score across its words, and use it as your "summary." Apply this to 10 articles and print your extractive summary next to the dataset's reference summary.

Q12.2. Using a Hugging Face pipeline("summarization") with a pretrained **abstractive** model, generate summaries for the same 10 articles. Compare qualitatively (in a short table or paragraph per example) to both your extractive baseline and the reference summaries.

Q12.3. Explain, in your own words, the difference between extractive and abstractive summarization, and give one advantage and one disadvantage of each approach based on what you observed in Q12.1 and Q12.2.

Q12.4. Compute the average compression ratio (summary length / article length, in words) for the dataset's reference summaries, grouped by category. Is the compression ratio consistent across categories, or does it vary? Suggest one reason for any variation you find.

Part 13: Spam Detection

Dataset file(s): `spam_detection.csv` | Points: 18

3,000 messages labeled Spam or Ham across 7 message categories (advertisements, scams, work messages, personal notes, etc.), with no real personal or financial data included.

Q13.1. Plot the distribution of `label` (Spam/Ham) and `category`. Is the dataset balanced? If not, explain one technique you could use to handle the imbalance during model training.

Q13.2. Train a TF-IDF + classifier pipeline (your choice of algorithm) to predict `label` from `text`. Report accuracy, precision, recall, and F1-score, and explain which metric matters most for a spam filter and why (consider the cost of false positives vs. false negatives).

Q13.3. Extract and display the top 15 words most strongly associated with the Spam class based on your trained model's feature weights. Do these words match your intuition about what spam "looks like"?

Q13.4. Test your trained model on 5 sentences you write yourself (not from the dataset) — include at least one borderline case that could plausibly be either spam or ham. Report your model's predictions and discuss whether they match your expectations.

Q13.5. Discuss (4-5 sentences): this dataset's spam messages follow fairly predictable templates. What challenges would you expect a spam classifier trained only on this data to face if deployed on real-world email or SMS messages, where spammers actively try to evade detection?

Part 14: Topic Modeling

Dataset file(s): `topic_modeling.csv` | Points: 18

3,000 documents evenly distributed across 10 technology/industry topics with relevant keyword annotations.

Q14.1. Using scikit-learn or Gensim, fit a Latent Dirichlet Allocation (LDA) model with $k=10$ topics on the `text` column. For each discovered topic, print its top 10 keywords.

Q14.2. Manually map each of your 10 discovered LDA topics to the closest matching ground-truth topic label based on the keywords. How well do your discovered topics align with the true labels? Are any ground-truth topics split across multiple LDA topics, or merged together? Explain what might cause this.

Q14.3. For each document, get its dominant LDA topic (the topic with the highest probability) and compute what fraction of documents were assigned to a topic matching their true label (using your mapping from Q14.2). Report this as an approximate accuracy score.

Q14.4. Repeat topic modeling with Non-negative Matrix Factorization (NMF) instead of LDA, again with $k=10$. Compare the top keywords per topic against your LDA results. Which method produced more interpretable topics for this dataset, in your judgment, and why?

Part 15: Language Detection

Dataset file(s): `language_detection.csv` | Points: 15

2,000 simple educational sentences evenly spread across 9 languages (English, French, Spanish, German, Italian, Portuguese, Arabic, Hindi, Urdu) with ISO-style language codes.

Q15.1. Plot the distribution of the `Language` column to confirm the classes are balanced.

Q15.2. Build a language classifier using **character-level** n-grams (e.g. `CountVectorizer(analyzer="char", ngram_range=(1, 3))`) combined with a classifier of your choice, to predict language from text. Report accuracy and a confusion matrix.

Q15.3. Rebuild the same classifier but using **word-level** features instead of character-level n-grams. Compare accuracy between the two approaches and explain, based on how these languages differ from each other (e.g. shared alphabets, distinct scripts), why character-level features are typically more robust for language identification.

Q15.4. Find the language pairs your classifier confuses most often (from the confusion matrix). Propose one linguistic reason why those particular languages might be harder to distinguish than others in this dataset.

Capstone: Integration Task

This final task asks you to combine skills from multiple parts of the assignment into a single mini pipeline. Choose **one** of the two options below.

Option A: End-to-End Review Analysis System

- Combine `sentiment_analysis.csv` and `named_entity_recognition.csv` conceptually: build a pipeline that, given a new piece of review-style text, (1) preprocesses it (Part 2), (2) predicts its sentiment and emotion using a model trained on `sentiment_analysis.csv` (Part 5), and (3) extracts any named entities it contains, using either your NER approach from Part 6 or spaCy.
- Test your full pipeline on 5 review sentences you write yourself, each mentioning at least one organization, product, or person name. For each, output a structured result (e.g. a dictionary with sentiment, emotion, and entities found).
- Write a short report (200-300 words) describing your pipeline's architecture, its accuracy/limitations, and one realistic business use case for a system like this (e.g. monitoring product reviews at scale).

Option B: Multi-Task Document Classifier & Explorer

- Combine `text_classification.csv`, `topic_modeling.csv`, and `text_summarization.csv`: build a pipeline that, given a new document, (1) predicts its category using your Part 7 classifier, (2) assigns it a dominant topic using your Part 14 LDA/NMF model, and (3) produces a one-sentence extractive summary using your Part 12 approach.
- Test your pipeline on 5 short documents you write yourself, each on a different topic. Output a structured result (predicted category, predicted topic, generated summary) for each.
- Write a short report (200-300 words) reflecting on where your pipeline's three components agreed or disagreed with each other, and what that tells you about the relationship between classification and topic modeling as two different ways of organizing the same text.

Submission Checklist

- Notebook(s) with all code cells executed and outputs visible (do not submit a cleared notebook).
- A short written interpretation beneath every question that asks for one.
- All plots titled and labeled.
- The capstone report (Option A or B) included as a Markdown cell or separate short document.
- A brief README or intro cell stating which libraries and model versions you used.

Dataset source: <https://www.kaggle.com/datasets/engraamirjamil/nlp-learning-dataset> — "NLP Learning Dataset" (synthetic, educational, CC0-licensed).